Exam Automata
June 23, 2025, 14:30 – 15:45, R10
Teacher: Hans van Heumen
Score = minimum(points obtained, 60)
Allowed resources:
No communication means like telephone, smartwatch, ...
But typically: laptop with EFIT + java + antlr + (offline) Javadoc + Python

Each question must be submitted in a **separate folder**.

General remarks:

- if you think that the questions are not precise enough to solve it, then you can add your own assumptions and continue your work.
  Please clearly write down your assumptions and the reason for adding it
- use the common rules for Separation of Concerns, such as: divide the tasks between the lexer, the parser, and the java/python actions; and avoid that a visitor does the tasks which should be done by its child visitors

1. Binary tree (40p)

To hand in:

a. Grammar file (`.g4`)
   - Ensure that all rules and tokens have meaningful names.
b. Source code (`.java` or `.py`)
c. Input files (`.txt`)
   - Should clearly demonstrate the capabilities of your grammar, i.e., it must include your own test cases.
d. Output files with clear, meaningful filename
e. Parse tree image (`.svg`)
f. (Optional) Additional documentation that clarifies or motivates your design decisions, in case the files above are not self-explanatory.

```
grammar ExamGrammar;

prog: stmt+ EOF;

stmt
    : printStmt ';'          # StatementPrint
    | assignment ';'         # StatementAssignment
    | expr ';'              # ExprStat
    ;

printStmt
    : 'print' expr           # PrintExpr
    ;

assignment
    : ID '=' expr            # AssignVar
    ;

expr
    : primaryExpr            # ExprBase
    | expr ('*' | '/') expr  # MulDiv
    | expr ('+' | '-') expr  # AddSub
    | ('+' | '-') expr       # Unary
    ;


primaryExpr
    : INT                    # Int
    | ID                     # Id
    | LPAREN expr RPAREN     # Parens
    | tree                   # treeLiteral
    ;

tree
    : /* TO BE COMPLETED BY STUDENT */
    ;

LPAREN : '(' ;
RPAREN : ')' ;
LBRACKET: '[' ;
RBRACKET: ']' ;
INT : [0-9]+;
ID  : [a-zA-Z_][a-zA-Z0-9_]*;
WS  : [ \t\r\n]+ -> skip;
```

Table 1

This assignment is about extending above grammar with binary trees. Binary trees are expressed according to the following samples:

1. **Empty tree:** t = []
2. **Single node tree:** t = [5 [] []]
3. **Root with one left child:** t = [6 [2 [] []] []]
4. **Root with two children and left child 2 has its own left child 4:** t = [1 [2 [4 [] []] []] [3 [] []]]

Assignment A — Extend the Grammar for Binary Trees (1p, 3p)

Q1. Extend the grammar from table 1 with *tree* rules, so that it supports binary tree literals as shown in the examples above. Show the new tree addition working with at least above four samples.

Q2. Extend your *tree* rule to allow any valid *expr* as the value of a node. For example, this should be accepted:

t = [1+2 [3*2 [] []] [6 [] []]];

Assignment B — Implement Tree Operations (3p, 4p, 3p, 3p, 4p)

Q1. Using the visitor pattern, implement code that builds a binary tree structure from the parsed input. Each node should store:

- an evaluated value (e.g., 1+2 becomes 3)
- a left and right subtree

Q2. Implement the *print* command. It should output the binary tree structure (from **Q1**) in the same bracketed format as used in the input samples

Q3. Extend the grammar with a unary operator treesum on a tree:

print treesum t;

This should print the sum of all evaluated node values in tree *t*.

Q4. Extend the grammar with a unary operator *treeheight* on a tree:

print treeheight t;

This should print the height (maximum depth) of binary tree *t*.

Q5. Extend the grammar with a unary operator getLeftTree on a tree:

print getLeftTree t;

This command should print the left subtree of binary tree t in the same bracketed format as used in the input samples. If the left child does not exist (e.g., for an empty tree [] or a node without a left child, like [5 [] []]), it should print [].

🖊 Assignment C — Tree Validation and Robustness (3p, 2p)

Q1. Ensure that both left and right subtrees can be empty, as in:

leaf = [42 [] []];

Demonstrate this by providing proof that your solution handles this case and similar structures correctly.

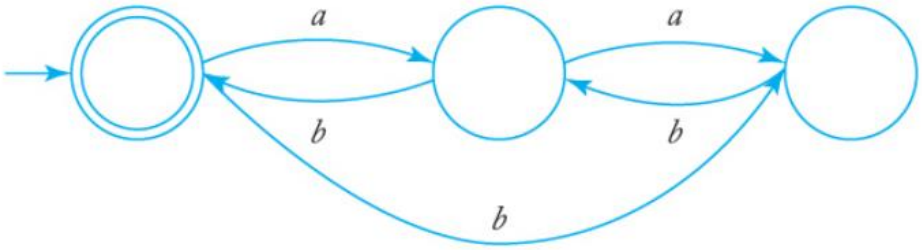
Q2. Ensure that expressions inside nodes are evaluated correctly during tree construction.
For example:
z = [1+1 [2+2 [] []] [3+3 [] []]];
should result in the structure:
[2 [4 [] []] [6 [] []]]

Demonstrate this by providing proof that your solution handles this case and similar structures correctly.

---

2. Regular expression ((2p, 3p), 5p)

1. Write regular expressions for the following languages on {0, 1}:
    a) All strings ending in 10.
    b) All strings not ending in 10
2. Find regular expressions for the languages accepted by the following automata.



Write your answer to a file in a separate folder.

3. Grammar (5p, 5p)

1. Find a regular grammar for the following language on {a, b}:
    $L = \{w : n_a(w) \geq 3, n_b(w) \text{ is odd}\}$.
    Note: $n_a(w)$* denotes the number of "a"s in the word w.
2. Find a context-free grammar for the language $L = \{a^n b^m : n = m - 2\}$.

Write your answer to a file in a separate folder.

4. DFA (5p)

Find dfa's for the following languages on {a, b}:

1. all the strings with exactly two a's and more than three b's.

Draw your solution on the official paper and hand in your paper.

-- end of exam --